

PRODUCT GUIDE

Sarathi

An agent that watches every government exam in India for one student, and stays quiet until something needs them.

What this is	A guide for anyone joining the project
Written for	Developers who have never seen this code
Frontend	Next.js 16, React 19, Tailwind v4
Backend	FastAPI, Python 3.14, PostgreSQL
Agents	Strands Agents SDK
Tests	206, all passing

1. What problem this solves

A student in India sits one government exam a year. They fight paperwork all year. Three things go wrong, and they stack on top of each other.

They do not know what exists. Thirty lakh people apply for SSC CGL. Meanwhile India Post GDS asks only for a 10th pass and holds no written exam, and far fewer people have heard of it.

They do not know if they qualify. Being eligible means five rules being true at the same time: your age, the extra years your category gives you, your qualification level, your home state, and how many attempts you have used. Most students find out they were never eligible only after paying the fee.

The paperwork takes it away. The photo must be between 20 and 50 KB. The correction window opens for five days and nobody announces it. A missed form costs a year of a life.

The one rule the whole project rests on. Every fact Sarathi states must come from a line in an official notification PDF. If it cannot find the line, it does not state the fact. It says it does not know.

This is why the product is worth building. Coaching websites and Telegram channels retype exam rules by hand and get them wrong. None of them show you where the rule came from. Sarathi shows the page number and the exact sentence, so a student can check it in ten seconds.

2. What it does, feature by feature

Feature	What it does for a student
Exam Radar	Shows every exam in four layers: central government, your state, your district, and other states open to everyone. Exams you cannot take are still shown, with the reason.
Result engine	Works out age with category relaxation, qualification level, marks, fee and home state. Six answers, including "closed for now, it runs again", because telling a student CGL is "not for you" would be false.
Education ladder	10th, 12th, ITI, Diploma, Graduation, Post graduation. Percentage or CGPA with live conversion. A diploma does not count as a degree.
Document maker	Turns one phone photo into the exact pixel size and file size each commission asks for. SSC wants a 236×79 signature, IBPS wants 140×60.
Document sheet	A single PDF with your details, pictures and marks, watermarked so nobody uploads it to a form by mistake.
Agent Journal	A record of every nightly run: what was checked, what was re-verified, and how many messages were sent. Usually zero.

Corrections	Lists corrections a commission published after the notification. Says plainly when a correction is a scan it cannot read.
Deadlines and age limit	Every date read from the notification, plus the birthday on which an exam closes to you permanently.
Fee savings	An SC candidate pays Rs 175 where others pay Rs 850. Most students never realise.
Two languages	English and Hindi. A test fails if either is missing.

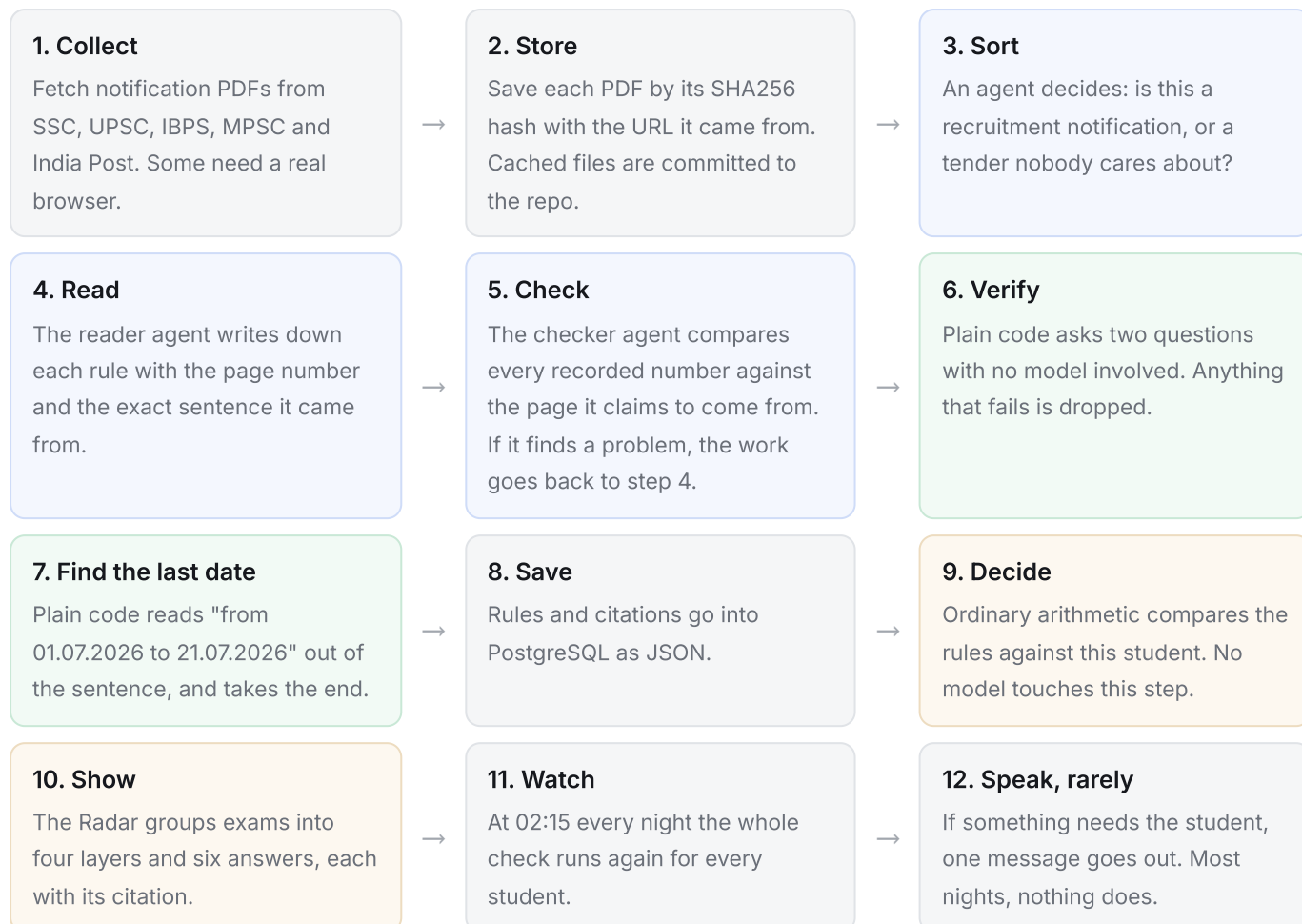
3. The tech stack, and why each piece

Piece	What we use	Why
Frontend	Next.js 16, React 19	Pages are rendered on the server, so data arrives with the page instead of being fetched by the browser.
Styling	Tailwind v4	Colours are defined once in <code>globals.css</code> and used everywhere.
Backend	FastAPI, Python 3.14	Types are checked at the edge, and the API documents itself at <code>/docs</code> .
Database	PostgreSQL, Neon free tier	Rules are stored as JSON with their citations attached.
Agents	Strands Agents SDK	Gives us tools, typed output, and a graph where one agent can send work back to another.
Model	Groq, or AWS Bedrock	One environment variable switches provider. The rest of the code never knows which is in use.
PDF reading	pypdf	Pulls text out of notification PDFs, page by page.
Browser	Playwright	MPSC and some SSC pages only appear after JavaScript runs.
Images	Pillow	Resizes photos to exact pixels and file size, and draws the PDF sheet.
File storage	ImageKit	Signatures are stored privately, behind links that expire in an hour.
Passwords	Argon2id	The current recommended way to store a password.

Everything runs on free tiers. Without ImageKit, documents save to local files. Without SMTP, verification codes go to `data/emails.log`. The project runs for anyone who clones it, with only a database and one model key.

4. How the system fits together

Read this top to bottom. It is the path a government PDF takes to become one sentence on a student's screen.



Why steps 5, 6 and 7 are separate

This is the most important design decision in the project, so it is worth being clear about.

During development we planted two errors in real extracted data. We changed a maximum age from 30 to 45, and an OBC relaxation from 3 years to 9. The AI verifier **missed both**. The plain numeric check **caught both instantly**.

The most dangerous mistake this product can make is a real quote paired with a wrong number, because the quote looks correct and a model reading it can be persuaded. Arithmetic cannot be persuaded. So the two checks that matter run in ordinary code:

1. **Is that sentence really on that page?** String matching.
2. **Is that number really inside that sentence?** Arithmetic.
3. *Anything the first two cannot see.* An agent, and only here.

5. How data moves

When a student opens the Radar

```
Browser -> Next.js server component
          reads the session cookie, calls the API with the token

Next.js -> GET /students/{id}/radar
          FastAPI loads the profile and every saved rule

FastAPI -> for each exam:
          is this exam open to your state?
          your age, plus the years your category adds
          your qualification level, then your marks
          the fee for your category
          each answer carries the citation it came from

FastAPI -> 17 exams, in 4 layers, with 6 possible answers
Browser -> "You get 3 extra years because you are OBC."
          page 6: "Other Backward Classes (Non-Creamy Layer) 3 years"
```

When a student uploads a photo

```
Browser -> POST /me/documents/photograph (the original file)
FastAPI -> check it is really an image, and big enough
          -> keep the original on disk as the master copy
          -> upload a copy to ImageKit
          -> save width, height and size in PostgreSQL

Later, when the Documents page is opened:
Next.js -> GET /me/documents/photograph/sizes
FastAPI -> for each commission, resize the master to that exact
          pixel size, then adjust quality until the file lands
          inside the KB range that commission allows
          -> return each one as base64

Browser -> four download buttons, one per commission
```

A detail worth copying

The sizes are built while the page is rendered on the server, not fetched by the browser afterwards. An earlier version hid them behind a button that fetched from the browser, and when that fetch failed the screen simply did not change. The student saw nothing and had no idea why. Building it on the server removed that whole class of failure.

6. Where everything lives

Backend

Folder	What is in it
app/sources/	One file per government website. Each knows how to find that site's notification PDFs.
app/storage/	The PDF cache. Files are stored by hash so the same document is never processed twice.
app/extraction/	Reading PDFs, checking citations, and the plain code that finds the last date to apply.
agents/	The Strands agents: reader, checker, classifier, verifier, and the graph that joins them.
app/eligibility/	The result engine. Age, categories, qualification levels, fees, layers. No models here at all.
app/student/	The student profile and the education ladder.
app/documents/	Image resizing, the per commission size rules, storage, and the PDF sheet.
app/journal/	The nightly run, the scheduler and the record of what happened.
app/auth/	Sign up, email codes, passwords, session tokens.
app/api/routes/	Every HTTP endpoint.
app/db/	Tables and the functions that read and write them.
scripts/	Commands you run by hand: fetch notices, classify them, read one document, sync to the database.
tests/	206 tests. They can never send a real email or a real message.

Frontend

Path	What is in it
app/(public)/	Landing page, sign up, email code, profile setup, first photos.
app/(app)/	Everything behind sign in: dashboard, radar, deadlines, documents, corrections, journal, profile.
components/	Sidebar, top bar, the education ladder form, shared cards and buttons.

lib/

API calls, session reading, formatting.

app/globals.css

Every colour in the product. Read section 8 before you add one.

7. Running it on your machine

```
# Backend
cd backend
python -m venv .venv
.venv/Scripts/python -m pip install -r requirements.txt
.venv/Scripts/python -m playwright install chromium

cp .env.example .env          # fill in DATABASE_URL and GROQ_API_KEY
.venv/Scripts/python -m alembic upgrade head
.venv/Scripts/python -m uvicorn app.main:app --port 8020

# Frontend, in another terminal
cd frontend
npm install
cp .env.example .env.local    # point NEXT_PUBLIC_API_BASE at port 8020
npm run dev                   # opens on http://localhost:3000
```

Loading exam data

```
.venv/Scripts/python -m scripts.refresh_notices    # download official PDFs
.venv/Scripts/python -m scripts.classify_cached    # notification, or tender?
.venv/Scripts/python -m scripts.process_document "CRP-P0" # read the rules out
.venv/Scripts/python -m scripts.sync_to_db         # put them in PostgreSQL
```

Do not skip `sync_to_db` . Reading a document writes the rules to a JSON file on disk. Until you sync, the website will not show them. This has confused people more than once.

Tests

```
cd backend
.venv/Scripts/python -m pytest tests/ -q          # 206 tests, about 8 minutes
```

8. If you want to contribute

The most useful thing you can add is **another exam source**. The radar is thin, and every new source makes the product more useful straight away.

Adding a new commission, step by step

1. Create `app/sources/your_source.py` . Copy `india_post.py` , it is the shortest one.
2. Set `id` , `name` , `home_url` and `official_domains` . The domain list matters: it stops us following a link to some random website.
3. Write `fetch_notices()` . If the PDFs are plain links, `collect_pdf_notices()` does the work for you.
4. Filter out the noise. India Post publishes about two hundred shortlist PDFs and one real notification, so that source keeps only what is about applying.
5. Add your module to `SOURCES` in `app/sources/registry.py` .
6. Add the body name to `app/exams/naming.py` , so students see "India Post GDS" instead of "Descriptive Notification".
7. If the exam is state level, add it to `SOURCE_PROFILES` in `app/eligibility/layers.py` with its state and whether it needs domicile.
8. Run the four data commands above, then open the Radar.

The rules of this codebase

- **Never state a fact without a citation.** If the reader cannot find the sentence, the fact is dropped, not guessed.
- **Never say an exam is open unless you found the closing date.** Say you do not know and link to the official page.
- **No model decides eligibility.** Models read documents. Arithmetic decides.
- **Plain, direct writing everywhere in the product.** Say "result", not "verdict". Say "check", not "judge". One idea per sentence, no clever phrasing.
- **Small files, one job each.** No comments unless something is genuinely surprising.
- **Only use colours defined in `globals.css`** . A test enforces this. It exists because a download button once used a colour that did not exist, so it rendered as white text on a white card and was invisible for days.

9. What is not finished

Being honest about this is more useful than pretending otherwise.

Gap

What it means

Only six sources	SSC, UPSC, IBPS, MPSC, India Post and the SSC exam calendar. This is the biggest weakness. Usually only one or two exams are open at a time, so the Radar looks empty.
MPSC cannot be read	Their notifications are photographs of paper with no text inside. They appear with their real title, date and official link, marked as unread. Reading them needs a vision model.
Not deployed	Everything runs on one machine. Moving the nightly run to the cloud needs AWS credits.
Multi post adverts	UPSC advertisements list many posts with different rules each. Sarathi handles single rule notifications well and says so when it cannot read one cleanly.
Old journal rows	Runs recorded before 3 September counted messages that were never actually delivered. Newer runs are accurate.

Sarathi. Built for the AWS Agents for Humans hackathon, Everyday Agents track.

Regenerate this document with `python -m scripts.make_product_doc` .